ANALISIS Y DISEÑO DE ALGORITMOS

Sesión 03:

Algoritmos de Ordenación Externa

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo

Los **algoritmos de ordenación externa** se utilizan cuando los datos a ordenar no caben completamente en la memoria principal y deben almacenarse en dispositivos de almacenamiento secundarios (como discos duros o SSD). Estos algoritmos son comunes en bases de datos grandes o sistemas que manejan grandes volúmenes de datos.

Uno de los algoritmos más conocidos de ordenación externa es el **Algoritmo de Ordenación por Mezcla Externa (Merge Sort Externo)**. A continuación te doy un ejemplo de pseudocódigo para este algoritmo:

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo

Algoritmo: Ordenación por Mezcla Externa (Merge Sort Externo)

1. **Dividir** los datos en bloques más pequeños que caben en la memoria.
2. **Ordenar** cada bloque de datos individualmente utilizando un algoritmo de ordenación interno (como el **QuickSort** o **HeapSort**).
3. **Fusionar** los bloques ordenados de manera eficiente.

```
Algoritmo OrdenacionExterna(datos):
    # Paso 1: Dividir los datos en bloques más pequeños
    bloques = dividirEnBloques(datos)

    # Paso 2: Ordenar cada bloque individualmente
    para cada bloque en bloques:
        ordenarBloque(bloque)

    # Paso 3: Fusionar los bloques ordenados
    mientras la cantidad de bloques sea mayor que 1:
        bloquesFusionados = fusionarBloques(bloques)
        bloques = bloquesFusionados

    # El resultado es un solo bloque ordenado
    devolver bloques[0]
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo

```
Función dividirEnBloques(datos):
    # Divide los datos en bloques que caben en la memoria
    bloques = []
    tamañoBloque = tamañoMaximoMemoria
    para i desde 0 hasta longitud(datos) con paso tamañoBloque:
        bloque = datos[i:i+tamañoBloque]
        bloques.append(bloque)
    devolver bloques

Función ordenarBloque(bloque):
    # Ordenar el bloque utilizando un algoritmo de ordenación interno
    ordenar(bloque)  # Usualmente se usa QuickSort, HeapSort, etc.
```

```
Función fusionarBloques(bloques):
    # Fusionar los bloques de manera eficiente
    fusionados = []
    mientras bloques no estén vacíos:
        # Elegir el menor elemento de cada bloque (o el más adecuado dependiendo del algoritm
        mínimoElemento = obtenerMinimoDeLosBloques(bloques)
        fusionados.append(minimoElemento)
        actualizarBloques(bloques)  # Elimina el elemento que ya fue procesado
    devolver fusionados

Función obtenerMinimoDeLosBloques(bloques):
    # Devuelve el menor elemento de todos los bloques
    mínimoElemento = infinito
    bloqueSeleccionado = None
    para cada bloque en bloques:
        si el primer elemento del bloque es menor que mínimoElemento:
            mínimoElemento = primerElemento(bloque)
            bloqueSeleccionado = bloque
    devolver mínimoElemento
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo

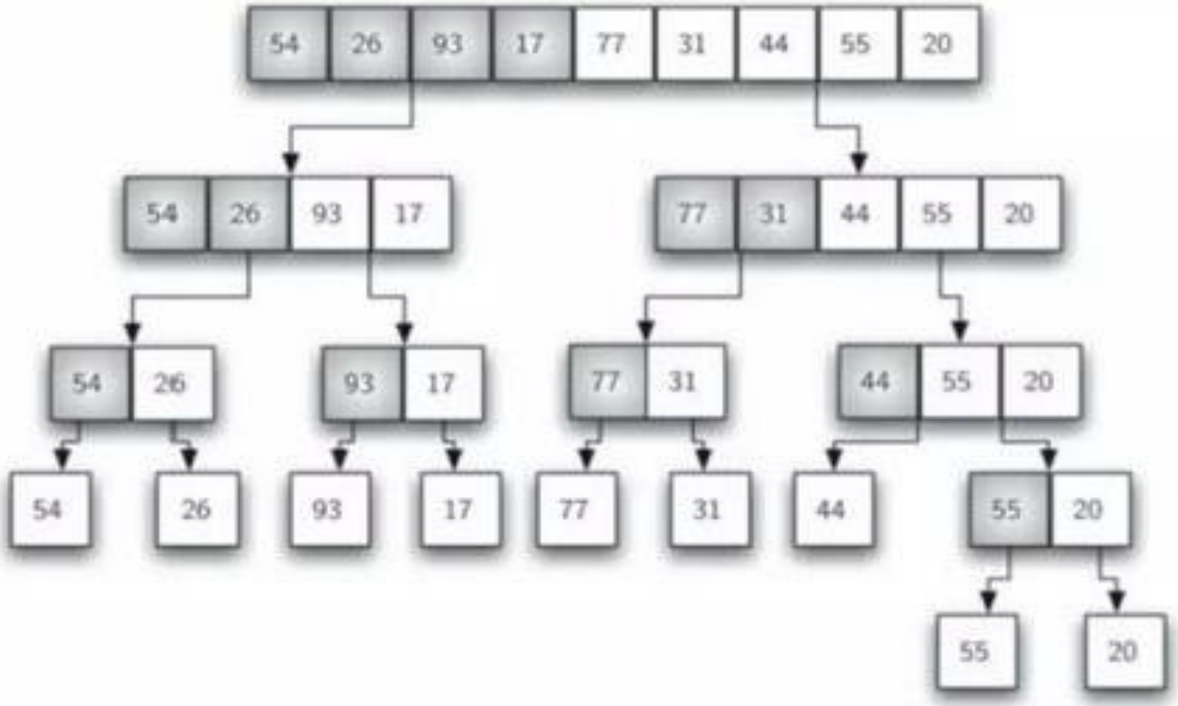
```
Función actualizarBloques(bloques):
    # Elimina el elemento mínimo ya procesado del bloque correspondiente
    para cada bloque en bloques:
        si primerElemento(bloque) es el mínimoElemento:
            eliminarPrimerElemento(bloque)
            romper
```

Explicación:

1. **División en bloques**: Primero, los datos se dividen en bloques más pequeños que pueden caber en la memoria, ya que no todos los datos caben en la memoria principal.
2. **Ordenación de bloques**: Cada bloque se ordena individualmente con un algoritmo eficiente (interno), ya que el tamaño del bloque es suficientemente pequeño para que se pueda ordenar completamente en memoria.
3. **Fusión de bloques**: Finalmente, los bloques ordenados se fusionan usando una técnica similar a la de la fusión en **Merge Sort**, en la que se toma el menor (o el más adecuado) elemento de cada bloque para formar una secuencia ordenada.

MEZCLA DIRECTA

El método Merge Sort consiste en partir una estructura en mitades, ordenar cada mitad y luego intercalar ordenadamente ambas mitades.



UTP Universidad Tecnológica del Perú

MEZCLA DIRECTA

Merge Sort Recursivo: Utiliza recursión para dividir el arreglo hasta que cada subarreglo tiene un solo elemento y luego los fusiona de manera ordenada.

Merge Sort Iterativo: Realiza el mismo proceso, pero utilizando un enfoque iterativo para fusionar los bloques, sin la necesidad de recursión.

Algoritmos de Ordenación Externa

Ejemplo1: Merge Sort Recursivo en Pseudocódigo

Este es el enfoque tradicional de Merge Sort, que utiliza recursión para dividir el arreglo hasta que cada parte tiene un solo elemento y luego las fusiona ordenadamente.

```
Algoritmo MergeSortRecursivo(arreglo)
    Si longitud(arreglo) > 1 entonces
        # Encuentra el punto medio
        medio <- longitud(arreglo) / 2

        # Divide el arreglo en dos mitades
        izquierda <- subarreglo(arreglo, 0, medio)
        derecha <- subarreglo(arreglo, medio, longitud(arreglo))

        # Llamadas recursivas para ordenar las dos mitades
        MergeSortRecursivo(izquierda)
        MergeSortRecursivo(derecha)

        # Fusiona las dos mitades ordenadas
        Fusionar(arreglo, izquierda, derecha)
    FinSi
FinAlgoritmo
```

```
Algoritmo Fusionar(arreglo, izquierda, derecha)
    i <- 0  # Índice para la izquierda
    j <- 0  # Índice para la derecha
    k <- 0  # Índice para el arreglo original

    Mientras i < longitud(izquierda) y j < longitud(derecha) hacer
        Si izquierda[i] < derecha[j] entonces
            arreglo[k] <- izquierda[i]
            i <- i + 1
        Sino
            arreglo[k] <- derecha[j]
            j <- j + 1
        FinSi
        k <- k + 1
    FinMientras

    # Copia los elementos restantes de izquierda, si los hay
    Mientras i < longitud(izquierda) hacer
        arreglo[k] <- izquierda[i]
        i <- i + 1
        k <- k + 1
    FinMientras

    # Copia los elementos restantes de izquierda, si los hay
    Mientras i < longitud(izquierda) hacer
        arreglo[k] <- izquierda[i]
        i <- i + 1
        k <- k + 1
    FinMientras

    # Copia los elementos restantes de derecha, si los hay
    Mientras j < longitud(derecha) hacer
        arreglo[k] <- derecha[j]
        j <- j + 1
        k <- k + 1
    FinMientras
FinAlgoritmo
```

Algoritmos de Ordenación Externa

Ejemplo2: Merge Sort Iterativo en Pseudocódigo

Este enfoque no utiliza recursión, sino que itera sobre el arreglo, fusionando bloques de manera gradual.

```
Algoritmo MergeSortIterativo(arreglo)
    n <- longitud(arreglo)

    # Comienza con bloques de tamaño 1 y los fusiona
    tamaño <- 1

    Mientras tamaño < n hacer
        # Fusiona bloques de tamaño 'tamaño'
        para i desde 0 hasta n - 1 con paso 2 * tamaño hacer
            izquierda <- subarreglo(arreglo, i, min(i + tamaño - 1, n - 1))
            derecha <- subarreglo(arreglo, i + tamaño, min(i + 2 * tamaño - 1, n - 1))
            Fusionar(arreglo, izquierda, derecha, i)
        FinPara

        # Duplica el tamaño del bloque
        tamaño <- 2 * tamaño
    FinMientras
FinAlgoritmo
```

```
Algoritmo Fusionar(arreglo, izquierda, derecha, inicio)
    i <- 0  # Índice para izquierda
    j <- 0  # Índice para derecha
    k <- inicio  # Índice para el arreglo original

    Mientras i < longitud(izquierda) y j < longitud(derecha) hacer
        Si izquierda[i] < derecha[j] entonces
            arreglo[k] <- izquierda[i]
            i <- i + 1
        Sino
            arreglo[k] <- derecha[j]
            j <- j + 1
        FinSi
        k <- k + 1
    FinMientras

    # Copia los elementos restantes de izquierda, si los hay
    Mientras i < longitud(izquierda) hacer
        arreglo[k] <- izquierda[i]
        i <- i + 1
        k <- k + 1
    FinMientras

    # Copia los elementos restantes de derecha, si los hay
    Mientras j < longitud(derecha) hacer
        arreglo[k] <- derecha[j]
        j <- j + 1
        k <- k + 1
    FinMientras
FinAlgoritmo
```

Algoritmos de Ordenación Externa

Ejemplo3: Algoritmo de Mezcla Directa (Merge Sort)

```
Algoritmo MergeSort(A)
    Si el tamaño de A > 1
        Dividir A en dos sublistas:
        listaIzquierda = A[0..mitad-1]
        listaDerecha = A[mitad..fin]

        Llamar recursivamente a MergeSort(listaIzquierda)
        Llamar recursivamente a MergeSort(listaDerecha)

        Llamar a Mezclar(listaIzquierda, listaDerecha) y devolver el resultado
    FinSi
FinAlgoritmo
```

Copiar

```
Algoritmo Mezclar(listaIzquierda, listaDerecha)
    Crear una lista vacía llamada resultado
    Mientras que listaIzquierda no esté vacía y listaDerecha no esté vacía
        Si el primer elemento de listaIzquierda es menor o igual que el primer elemento de li
            Insertar el primer elemento de listaIzquierda en resultado
            Eliminar el primer elemento de listaIzquierda
        Sino
            Insertar el primer elemento de listaDerecha en resultado
            Eliminar el primer elemento de listaDerecha
        FinSi
    FinMientras

    Si listaIzquierda no está vacía
        Agregar todos los elementos restantes de listaIzquierda a resultado
    SinoSi listaDerecha no está vacía
        Agregar todos los elementos restantes de listaDerecha a resultado
    FinSi

    Devolver resultado
FinAlgoritmo
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo4: Algoritmo de Mezcla Directa con manejo de índices

```
Algoritmo MergeSort(A, inicio, fin)
    Si inicio < fin
        mitad = (inicio + fin) / 2
        Llamar a MergeSort(A, inicio, mitad)
        Llamar a MergeSort(A, mitad + 1, fin)
        Llamar a Mezclar(A, inicio, mitad, fin)
    FinSi
FinAlgoritmo
```

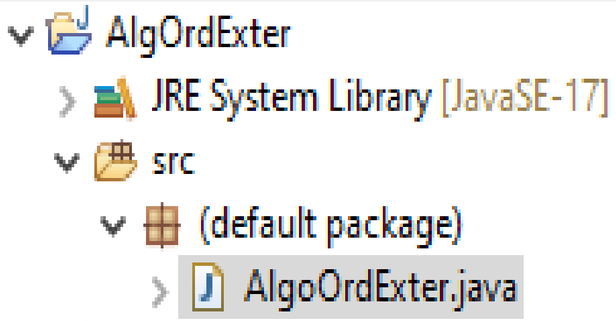
```
Algoritmo Mezclar(A, inicio, mitad, fin)
    Crear un arreglo temporal izquierdo[mitad - inicio + 1]
    Crear un arreglo temporal derecho[fin - mitad]

    Para i = 0 hasta mitad - inicio
        izquierdo[i] = A[inicio + i]
    FinPara

    Para j = 0 hasta fin - mitad - 1
        derecho[j] = A[mitad + 1 + j]
    FinPara
```

```
i = 0, j = 0, k = inicio
Mientras i < tamaño de izquierdo y j < tamaño de derecho
    Si izquierdo[i] <= derecho[j]
        A[k] = izquierdo[i]
        i = i + 1
    Sino
        A[k] = derecho[j]
        j = j + 1
    FinSi
    k = k + 1
FinMientras

Mientras i < tamaño de izquierdo
    A[k] = izquierdo[i]
    i = i + 1
    k = k + 1
FinMientras

Mientras j < tamaño de derecho
    A[k] = derecho[j]
    j = j + 1
    k = k + 1
FinMientras
FinAlgoritmo
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo1: Implementación Recursiva de Merge Sort

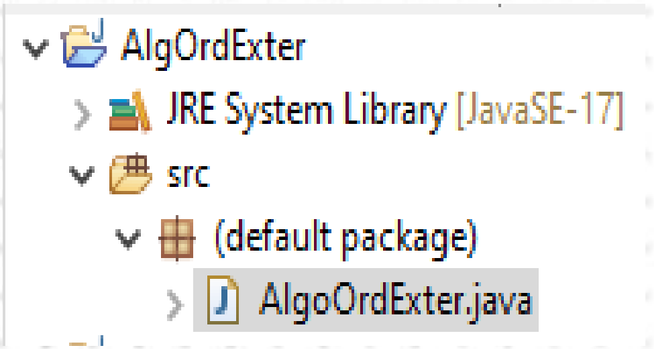



```java
import java.util.Arrays;

public class AlgoOrdExter {
      // Método principal que llama a mergeSort en el arreglo
    public static void main(String[] args) {
        int[] arr = {38, 27, 43, 3, 9, 82, 10};

        System.out.println("Arreglo original: " + Arrays.toString(arr));
        mergeSort(arr, 0, arr.length - 1);
        System.out.println("Arreglo ordenado: " + Arrays.toString(arr));
    }
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo1: Implementación Recursiva de Merge Sort




```java
// Método recursivo que divide el arreglo y ordena
public static void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int middle = (left + right) / 2; // Encuentra el punto medio

        // Divide el arreglo en dos mitades
        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);

        // Mezcla las dos mitades ordenadas
        merge(arr, left, middle, right);
    }
}
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo1: Implementación Recursiva de Merge Sort

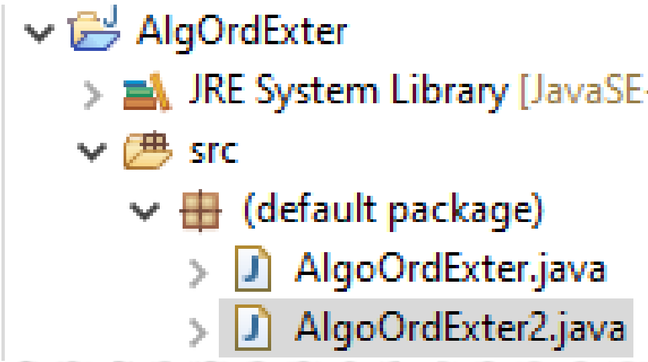
```java
// Método para fusionar dos mitades
public static void merge(int[] arr, int left, int middle, int right) {
    // Encuentra el tamaño de los subarreglos a fusionar
    int n1 = middle - left + 1;
    int n2 = right - middle;

    // Arreglos temporales para las mitades
    int[] leftArray = new int[n1];
    int[] rightArray = new int[n2];

    // Copia los datos a los arreglos temporales
    System.arraycopy(arr, left, leftArray, 0, n1);
    System.arraycopy(arr, middle + 1, rightArray, 0, n2);

    // Índices para los subarreglos y el arreglo original
    int i = 0, j = 0, k = left;

    // Fusión de los dos subarreglos
    while (i < n1 && j < n2) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        } else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }

    // Copia los elementos restantes, si los hay
    while (i < n1) {
        arr[k] = leftArray[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = rightArray[j];
        j++;
        k++;
    }
}
}
```

```
Arreglo original: [38, 27, 43, 3, 9, 82, 10]
Arreglo ordenado: [3, 9, 10, 27, 38, 43, 82]
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo2: Implementación Iterativa de Merge Sort




```java
import java.util.Arrays;


public class AlgoOrdExter2 {
    // Método principal
    public static void main(String[] args) {
        int[] arr = {38, 27, 43, 3, 9, 82, 10};

        System.out.println("Arreglo original: " + Arrays.toString(arr));
        mergeSort(arr);
        System.out.println("Arreglo ordenado: " + Arrays.toString(arr));
    }

  // Método iterativo de Merge Sort
  public static void mergeSort(int[] arr) {
      int n = arr.length;
      // Tamaño de los bloques a fusionar
      for (int size = 1; size < n; size = 2 * size) {
          for (int leftStart = 0; leftStart < n - 1; leftStart += 2 * size) {
              // Encuentra el punto medio y el final del bloque derecho
              int mid = Math.min(n - 1, leftStart + size - 1);
              int rightEnd = Math.min(n - 1, leftStart + 2 * size - 1);

              // Fusiona los dos bloques
              merge(arr, leftStart, mid, rightEnd);
          }
      }
  }
```

Algoritmos de Ordenación Externa

Ejemplo2: Implementación Iterativa de Merge Sort

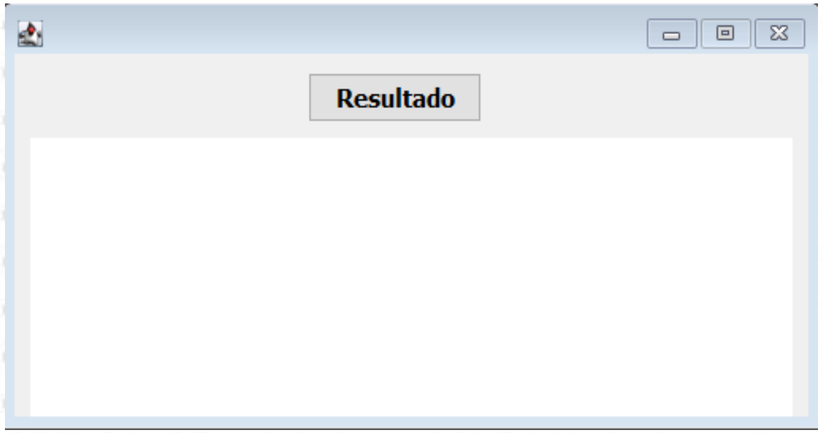
```java
// Método para fusionar dos subarreglos
public static void merge(int[] arr, int left, int mid, int right) {
    // Encuentra el tamaño de los subarreglos
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Arreglos temporales
    int[] leftArray = new int[n1];
    int[] rightArray = new int[n2];

    // Copia los datos en los arreglos temporales
    System.arraycopy(arr, left, leftArray, 0, n1);
    System.arraycopy(arr, mid + 1, rightArray, 0, n2);

    // Índices para los subarreglos y el arreglo original
    int i = 0, j = 0, k = left;

    // Fusiona los dos subarreglos
    while (i < n1 && j < n2) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        } else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }

    // Copia los elementos restantes del subarreglo izquierdo
    while (i < n1) {
        arr[k] = leftArray[i];
        i++;
        k++;
    }

    // Copia los elementos restantes del subarreglo derecho
    while (j < n2) {
        arr[k] = rightArray[j];
        j++;
        k++;
    }
}
}
```

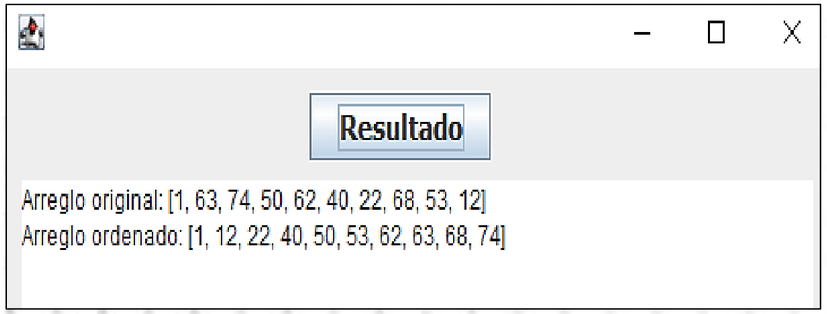
```
Arreglo original: [38, 27, 43, 3, 9, 82, 10]
Arreglo ordenado: [3, 9, 10, 27, 38, 43, 82]
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo1: Merge Sort Recursivo con JFrame






```java
public class AlgoOrdExter1Inter extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private int[] numbers;
```

```java
public void actionPerformed(ActionEvent e) {
    // Crear un arreglo de números aleatorios
    numbers = new Random().ints(10, 1, 100).toArray();
    textArea.setText("Arreglo original: " + Arrays.toString(numbers) + "\n");

    // Llamar al Merge Sort
    mergeSort(numbers, 0, numbers.length - 1);

    // Mostrar el arreglo ordenado
    textArea.append("Arreglo ordenado: " + Arrays.toString(numbers));
}
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo1: Merge Sort Recursivo con JFrame

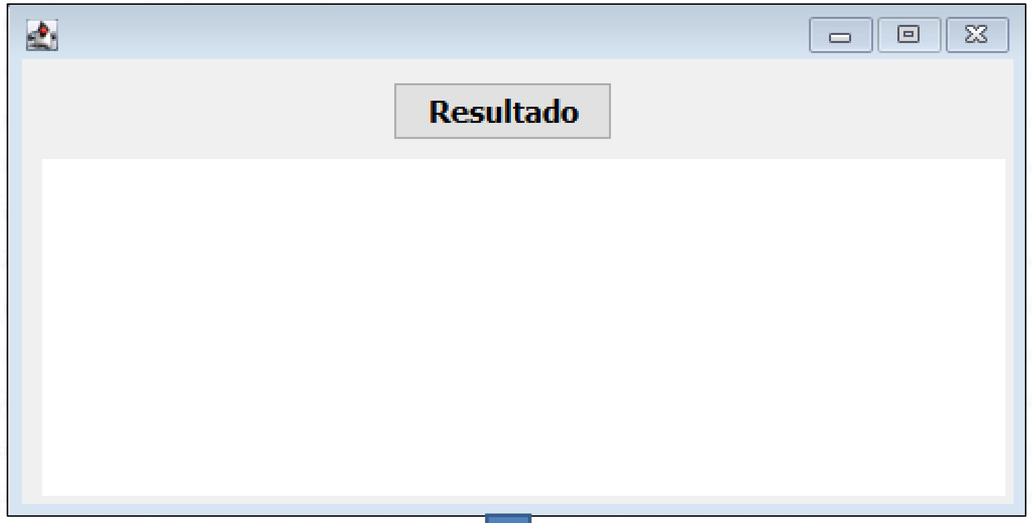
```java
  // Método recursivo de Merge Sort
public void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);        // Ordenar la mitad izquierda
        mergeSort(arr, mid + 1, right);   // Ordenar la mitad derecha
        merge(arr, left, mid, right);     // Fusionar las dos mitades
    }
}

// Método para fusionar dos subarreglos
public void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] leftArr = new int[n1];
    int[] rightArr = new int[n2];

    System.arraycopy(arr, left, leftArr, 0, n1);
    System.arraycopy(arr, mid + 1, rightArr, 0, n2);
```

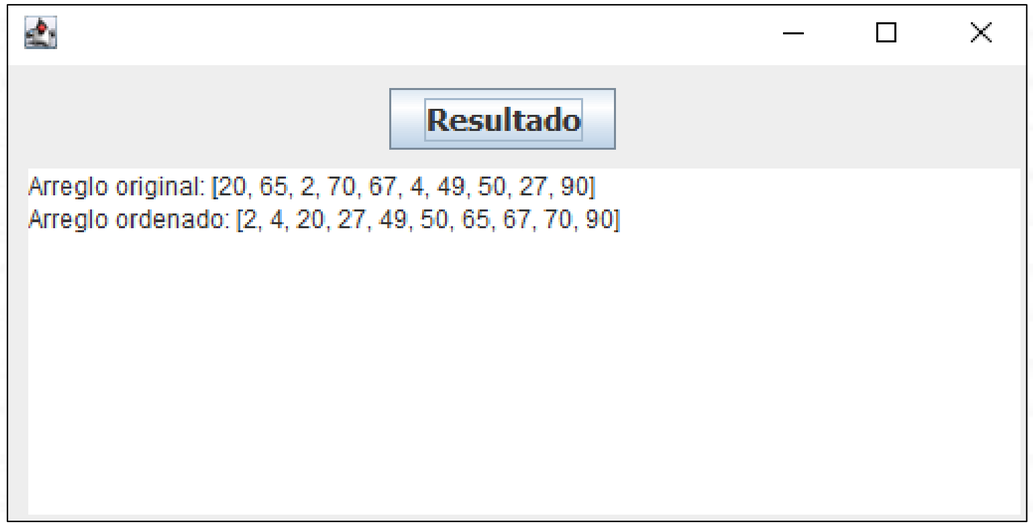
```java
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
    if (leftArr[i] <= rightArr[j]) {
        arr[k] = leftArr[i];
        i++;
    } else {
        arr[k] = rightArr[j];
        j++;
    }
    k++;
}

    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo2: Merge Sort Iterativo con JFrame






```java
private static final long serialVersionUID = 1L;
private JPanel contentPane;
private int[] numbers;

public void actionPerformed(ActionEvent e) {
    // Crear un arreglo de números aleatorios
    numbers = new Random().ints(10, 1, 100).toArray();
    textArea.setText("Arreglo original: " + Arrays.toString(numbers) + "\n");

    // Llamar al Merge Sort iterativo
    mergeSortIterativo(numbers);

    // Mostrar el arreglo ordenado
    textArea.append("Arreglo ordenado: " + Arrays.toString(numbers));
}
```

```java
// Método iterativo de Merge Sort
public void mergeSortIterativo(int[] arr) {
    int n = arr.length;
    for (int size = 1; size < n; size = 2 * size) {
        for (int leftStart = 0; leftStart < n - 1; leftStart += 2 * size) {
            int mid = Math.min(n - 1, leftStart + size - 1);
            int rightEnd = Math.min(n - 1, leftStart + 2 * size - 1);
            merge(arr, leftStart, mid, rightEnd);
        }
    }
}
```

Ejemplo2: Merge Sort Iterativo con JFrame

```java
// Método para fusionar dos subarreglos
public void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] leftArr = new int[n1];
    int[] rightArr = new int[n2];

    System.arraycopy(arr, left, leftArr, 0, n1);
    System.arraycopy(arr, mid + 1, rightArr, 0, n2);
```

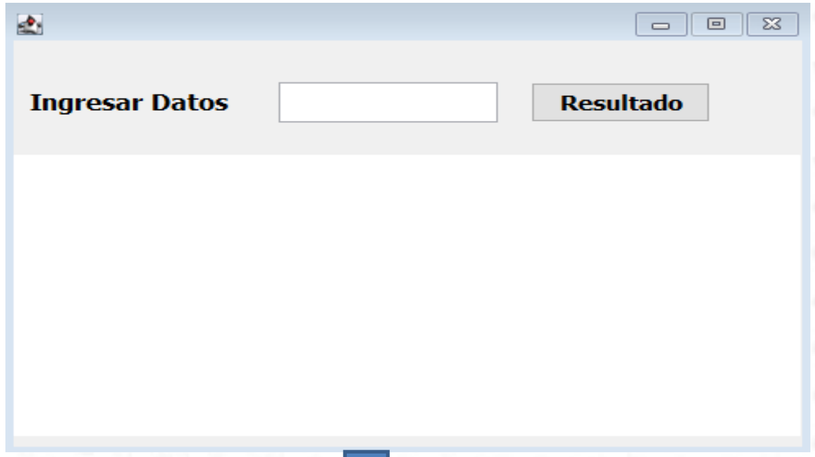
```java
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        } else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
```

```java
    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo1: Merge Sort Recursivo con Datos Ingresados por el Usuario



```java
public class AlgoOrdExter1Usuario extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField txtDatos;
    private int[] numbers;


public void actionPerformed(ActionEvent e) {
    String input = txtDatos.getText();
    try {
        // Convierte la entrada en un arreglo de enteros
        String[] inputArray = input.split(",");
        numbers = new int[inputArray.length];
        for (int i = 0; i < inputArray.length; i++) {
            numbers[i] = Integer.parseInt(inputArray[i].trim());
        }

        // Muestra el arreglo original
        textArea.setText("Arreglo original: " + Arrays.toString(numbers) + "\n")

        // Llama al algoritmo de Merge Sort recursivo
        mergeSort(numbers, 0, numbers.length - 1);

        // Muestra el arreglo ordenado
        textArea.append("Arreglo ordenado: " + Arrays.toString(numbers));
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(null, "Por favor ingresa solo números "
                + "válidos separados por comas.");
    }
}
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo1: Merge Sort Recursivo con Datos Ingresados por el Usuario

```java
// Método recursivo de Merge Sort
public void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);        // Ordenar la mitad izquierda
        mergeSort(arr, mid + 1, right);   // Ordenar la mitad derecha
        merge(arr, left, mid, right);     // Fusionar las dos mitades
    }
}
```

```java
// Método para fusionar dos subarreglos
public void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] leftArr = new int[n1];
    int[] rightArr = new int[n2];

    System.arraycopy(arr, left, leftArr, 0, n1);
    System.arraycopy(arr, mid + 1, rightArr, 0, n2);

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (leftArr[i] <= rightArr[j]) {
                arr[k] = leftArr[i];
                i++;
            } else {
                arr[k] = rightArr[j];
                j++;
            }
            k++;
        }
```

```java
        while (i < n1) {
            arr[k] = leftArr[i];
            i++;
            k++;
        }

        while (j < n2) {
            arr[k] = rightArr[j];
            j++;
            k++;
        }
}
```

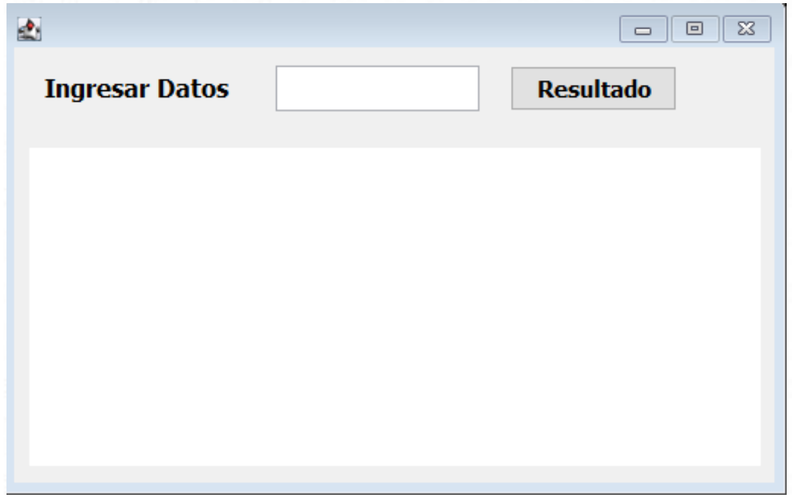
Explicación:

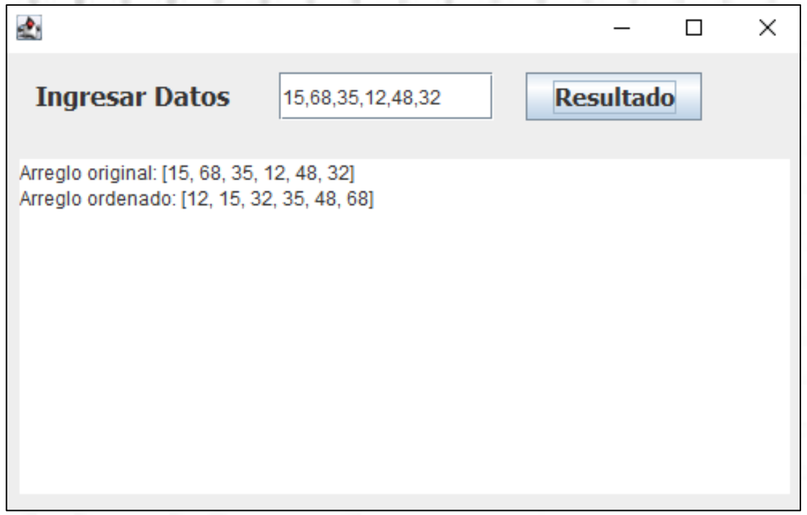
1. **Entrada del usuario**: El usuario ingresa una lista de números separados por comas en el campo de texto inputField.
2. **Botón de Ordenar**: Al presionar el botón "Ordenar", los números ingresados son leídos, convertidos en un arreglo de enteros, y luego ordenados usando el algoritmo **Merge Sort** recursivo.
3. **Mostrar resultados**: El JTextArea muestra el arreglo original y el arreglo ordenado después de aplicar el algoritmo.

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo2: Merge Sort Iterativo con Datos Ingresados por el Usuario






```java
public class AlgoOrdExter2Usuario extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField txtDatos;
    private int[] numbers;
```

```java
public void actionPerformed(ActionEvent e) {
    String input = txtDatos.getText();
        try {
            // Convierte la entrada en un arreglo de enteros
            String[] inputArray = input.split(",");
            numbers = new int[inputArray.length];
            for (int i = 0; i < inputArray.length; i++) {
                numbers[i] = Integer.parseInt(inputArray[i].trim());
            }

            // Muestra el arreglo original
            textArea.setText("Arreglo original: " + Arrays.toString(numbers) + "\n");

        // Muestra el arreglo original
        textArea.setText("Arreglo original: " + Arrays.toString(numbers) + "\n");

        // Llama al algoritmo de Merge Sort iterativo
        mergeSortIterativo(numbers);

        // Muestra el arreglo ordenado
        textArea.append("Arreglo ordenado: " + Arrays.toString(numbers));
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(null, "Por favor ingresa solo "
                + "números válidos separados por comas.");
    }
```

Algoritmos de Ordenación Externa

Ejemplo2: Merge Sort Iterativo con Datos Ingresados por el Usuario

```java
// Método iterativo de Merge Sort
public void mergeSortIterativo(int[] arr) {
    int n = arr.length;
    for (int size = 1; size < n; size = 2 * size) {
        for (int leftStart = 0; leftStart < n - 1; leftStart += 2 * size) {
            int mid = Math.min(n - 1, leftStart + size - 1);
            int rightEnd = Math.min(n - 1, leftStart + 2 * size - 1);
            merge(arr, leftStart, mid, rightEnd);
        }
    }
}
```

```java
// Método para fusionar dos subarreglos
public void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] leftArr = new int[n1];
    int[] rightArr = new int[n2];

    System.arraycopy(arr, left, leftArr, 0, n1);
    System.arraycopy(arr, mid + 1, rightArr, 0, n2);
```

```java
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
    if (leftArr[i] <= rightArr[j]) {
        arr[k] = leftArr[i];
        i++;
    } else {
        arr[k] = rightArr[j];
        j++;
    }
    k++;
}

    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}
```

Universidad Tecnológica del Perú

MEZCLA NATURAL

EL MÉTODO DE MEZCLA NATURAL CONSISTE EN APROVECHAR LA EXISTENCIA DE SECUENCIAS YA ORDENADAS.

- OPTIMIZACIÓN DE LA MEZCLA DIRECTA
- A PARTIR DE LAS SECUENCIAS ORDENADAS EXISTENTES EN EL ARCHIVO, SE OBTIENEN PARTICIONES QUE SE ALMACENAN EN FICHEROS AUXILIARES.

MEZCLA NATURAL

- Es mas eficiente que la mezcla directa debido a un menor tiempo de ejecucion.
- El algoritmo de mezcla natural aprovecha la existencia de secuencia ya ordenadas dentro de los datos del archivo.
- Partiendo de secuencias ordenadas obtenemos particiones que guardamos en dos archivos o ficheros auxiliares.
- Las particiones almacenadas se fusionan posteriormente para crear secuencias ordenadas cuya longitud se incrementa arbitrariamente hasta conseguir la total ordenación de los datos contenidos en el archivo original.

Algoritmos de Ordenación Externa

Ejemplo 1: Algoritmo de Mezcla Natural (Pseudocódigo)

```
Algoritmo MezclaNatural(A)
    Definir n como la longitud de A
    Si n <= 1
        retornar A

    // Paso 1: Dividir el arreglo en subsecuencias naturales
    Definir subsecuencias como lista vacía
    i = 0
    Mientras i < n
        j = i
        Mientras j < n - 1 y A[j] <= A[j + 1]
            j = j + 1
        Fin Mientras
        subsecuencias.agregar(A[i..j])
        i = j + 1
    Fin Mientras
```

```
    // Paso 2: Mezclar las subsecuencias naturales
    Mientras longitud de subsecuencias > 1
        subseq1 = subsecuencias.quitarPrimerElemento()
        subseq2 = subsecuencias.quitarPrimerElemento()
        nuevaSubsecuencia = Mezclar(subseq1, subseq2)
        subsecuencias.agregar(nuevaSubsecuencia)
    Fin Mientras

    retornar subsecuencias[0]
Fin Algoritmo
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo 1: Algoritmo de Mezcla Natural (Pseudocódigo)

```
Algoritmo Mezclar(subseq1, subseq2)
    Definir i = 0, j = 0
    Definir resultado como lista vacía
    Mientras i < longitud(subseq1) y j < longitud(subseq2)
        Si subseq1[i] <= subseq2[j]
            resultado.agregar(subseq1[i])
            i = i + 1
        Sino
            resultado.agregar(subseq2[j])
            j = j + 1
    Fin Mientras
```

```
    // Agregar los elementos restantes
    Mientras i < longitud(subseq1)
        resultado.agregar(subseq1[i])
        i = i + 1
    Fin Mientras
    Mientras j < longitud(subseq2)
        resultado.agregar(subseq2[j])
        j = j + 1
    Fin Mientras

    retornar resultado
Fin Algoritmo
```

Universidad Tecnológica del Perú

Ejemplo 2: Algoritmo de Mezcla Natural con Identificación de Subsecuencias

```
Algoritmo MezclaNatural(A)
    n = longitud(A)
    Si n <= 1
        retornar A

    // Paso 1: Identificar subsecuencias naturales
    subsecuencias = []
    inicio = 0
    Mientras inicio < n
        fin = inicio
        Mientras fin < n - 1 y A[fin] <= A[fin + 1]
            fin = fin + 1
        Fin Mientras
        subsecuencias.agregar(A[inicio..fin])
        inicio = fin + 1
    Fin Mientras
```

```
    // Paso 2: Mezclar las subsecuencias
    Mientras longitud(subsecuencias) > 1
        subseq1 = subsecuencias.quitarPrimerElemento()
        subseq2 = subsecuencias.quitarPrimerElemento()
        mezcla = Mezclar(subseq1, subseq2)
        subsecuencias.agregar(mezcla)
    Fin Mientras

    retornar subsecuencias[0]
Fin Algoritmo
```

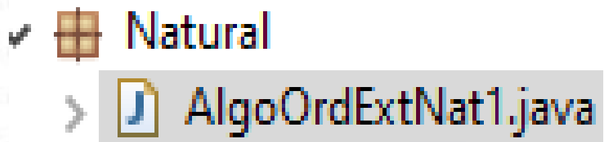
Ejemplo 2: Algoritmo de Mezcla Natural con Identificación de Subsecuencias

```
Algoritmo Mezclar(subseq1, subseq2)
    i = 0, j = 0
    resultado = []
    Mientras i < longitud(subseq1) y j < longitud(subseq2)
        Si subseq1[i] <= subseq2[j]
            resultado.agregar(subseq1[i])
            i = i + 1
        Sino
            resultado.agregar(subseq2[j])
            j = j + 1
    Fin Mientras
```

```
    // Agregar los elementos restantes
    Mientras i < longitud(subseq1)
        resultado.agregar(subseq1[i])
        i = i + 1
    Fin Mientras
    Mientras j < longitud(subseq2)
        resultado.agregar(subseq2[j])
        j = j + 1
    Fin Mientras

    retornar resultado
Fin Algoritmo
```

Algoritmos de Ordenación Externa

Ejemplo 1: Algoritmo de Mezcla Natural en Java




```java
package Natural;

import java.util.ArrayList;
import java.util.List;

public class AlgoOrdExtNat1 {

    public static void main(String[] args) {
        int[] A = {5, 8, 1, 3, 7, 6, 2, 4};
        naturalMergeSort(A);
        for (int i : A) {
            System.out.print(i + " ");
        }
    }
```

```java
// Función principal que realiza el algoritmo de mezcla natural
public static void naturalMergeSort(int[] A) {
    List<int[]> subsecuencias = new ArrayList<>();
    int n = A.length;

    // Paso 1: Identificar subsecuencias naturales
    int i = 0;
    while (i < n) {
        int j = i;
        while (j < n - 1 && A[j] <= A[j + 1]) {
            j++;
        }
        int[] subsecuencia = new int[j - i + 1];
        System.arraycopy(A, i, subsecuencia, 0, j - i + 1);
        subsecuencias.add(subsecuencia);
        i = j + 1;
    }

    // Paso 2: Mezclar las subsecuencias
    while (subsecuencias.size() > 1) {
        List<int[]> nuevasSubsecuencias = new ArrayList<>();
        for (int k = 0; k < subsecuencias.size(); k += 2) {
            if (k + 1 < subsecuencias.size()) {
                nuevasSubsecuencias.add(merge(subsecuencias.get(k),
                        subsecuencias.get(k + 1)));
            } else {
                nuevasSubsecuencias.add(subsecuencias.get(k));
            }
        }
        subsecuencias = nuevasSubsecuencias;
    }

    // Reemplazar el arreglo original con el resultado final
    System.arraycopy(subsecuencias.get(0), 0, A, 0, n);
}
```

Algoritmos de Ordenación Externa

Ejemplo 1: Algoritmo de Mezcla Natural en Java

- Natural
  - AlgoOrdExtNat1.java

```java
// Función para mezclar dos subsecuencias ordenadas
public static int[] merge(int[] subseq1, int[] subseq2) {
    int[] resultado = new int[subseq1.length + subseq2.length];
    int i = 0, j = 0, k = 0;

    while (i < subseq1.length && j < subseq2.length) {
        if (subseq1[i] <= subseq2[j]) {
            resultado[k++] = subseq1[i++];
        } else {
            resultado[k++] = subseq2[j++];
        }
    }

    // Agregar los elementos restantes
    while (i < subseq1.length) {
        resultado[k++] = subseq1[i++];
    }

    while (j < subseq2.length) {
        resultado[k++] = subseq2[j++];
    }

    return resultado;
}
}
```

1 2 3 4 5 6 7 8

UTP Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo 2: Algoritmo de Mezcla Natural en Java con Listas

- Natural
  - AlgoOrdExtNat1.java
  - AlgoOrdExtNat2.java

```java
package Natural;
import java.util.LinkedList;
import java.util.List;


public class AlgoOrdExtNat2 {
    public static void main(String[] args) {
        int[] A = {5, 8, 1, 3, 7, 6, 2, 4};
        naturalMergeSort(A);
        for (int i : A) {
            System.out.print(i + " ");
        }
    }
```

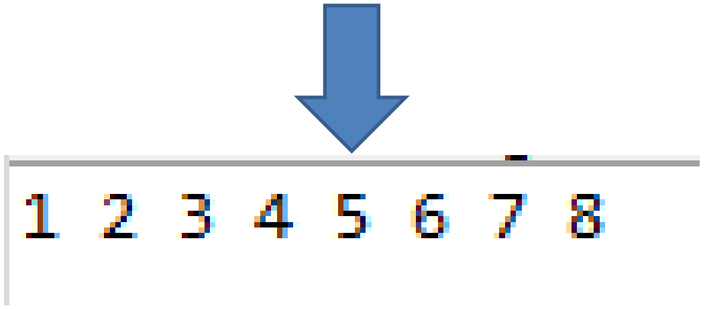
```java
// Función principal que realiza el algoritmo de mezcla natural
public static void naturalMergeSort(int[] A) {
    List<int[]> subsecuencias = new LinkedList<>();
    int n = A.length;

    // Paso 1: Identificar subsecuencias naturales
    int i = 0;
    while (i < n) {
        int j = i;
        while (j < n - 1 && A[j] <= A[j + 1]) {
            j++;
        }
        int[] subsecuencia = new int[j - i + 1];
        System.arraycopy(A, i, subsecuencia, 0, j - i + 1);
        subsecuencias.add(subsecuencia);
        i = j + 1;
    }


    // Paso 2: Mezclar las subsecuencias
    while (subsecuencias.size() > 1) {
        List<int[]> nuevasSubsecuencias = new LinkedList<>();
        for (int k = 0; k < subsecuencias.size(); k += 2) {
            if (k + 1 < subsecuencias.size()) {
                nuevasSubsecuencias.add(merge(subsecuencias.get(k),
                        subsecuencias.get(k + 1)));
            } else {
                nuevasSubsecuencias.add(subsecuencias.get(k));
            }
        }
        subsecuencias = nuevasSubsecuencias;
    }

    // Reemplazar el arreglo original con el resultado final
    System.arraycopy(subsecuencias.get(0), 0, A, 0, n);
}
```

Algoritmos de Ordenación Externa

Ejemplo 2: Algoritmo de Mezcla Natural en Java con Listas

```java
// Función para mezclar dos subsecuencias ordenadas
public static int[] merge(int[] subseq1, int[] subseq2) {
    int[] resultado = new int[subseq1.length + subseq2.length];
    int i = 0, j = 0, k = 0;

    while (i < subseq1.length && j < subseq2.length) {
        if (subseq1[i] <= subseq2[j]) {
            resultado[k++] = subseq1[i++];
        } else {
            resultado[k++] = subseq2[j++];
        }
    }

    // Agregar los elementos restantes
    while (i < subseq1.length) {
        resultado[k++] = subseq1[i++];
    }
```

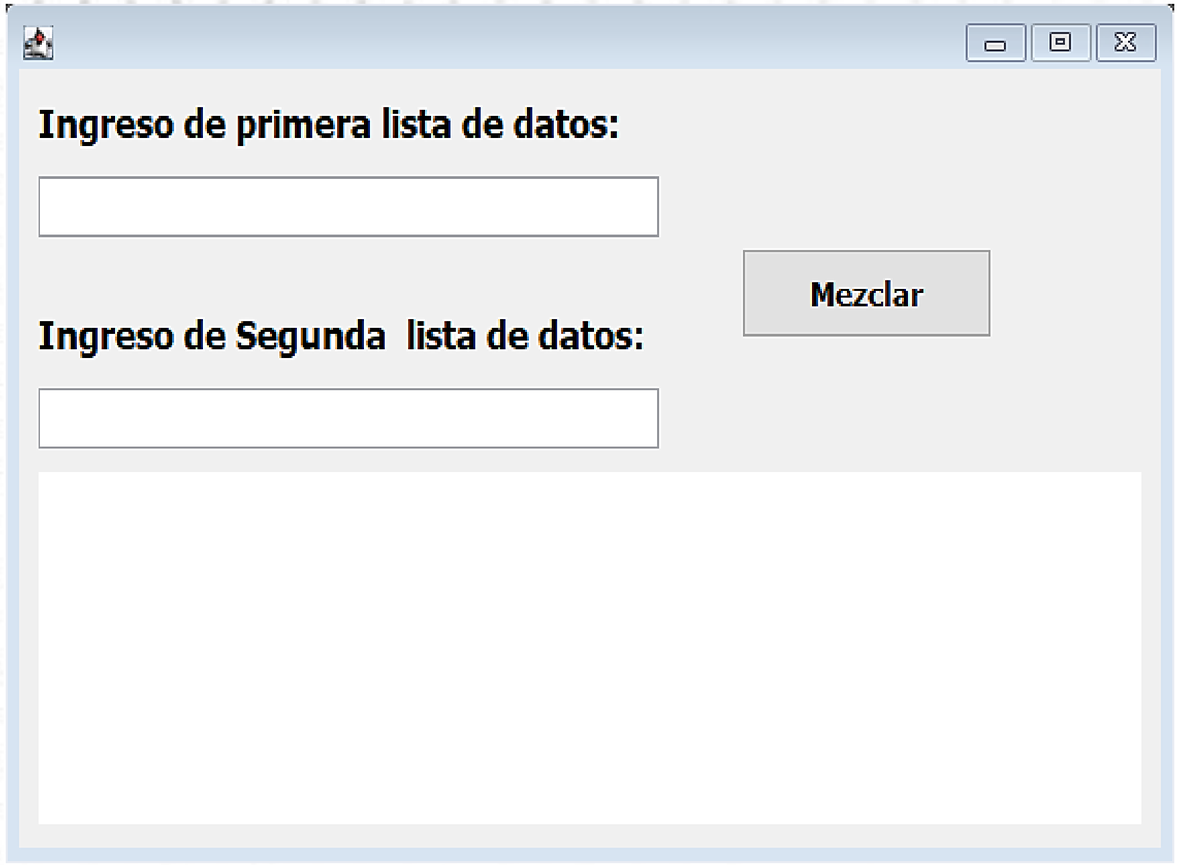
```java
    // Agregar los elementos restantes
    while (i < subseq1.length) {
        resultado[k++] = subseq1[i++];
    }

    while (j < subseq2.length) {
        resultado[k++] = subseq2[j++];
    }

    return resultado;
}
}
```

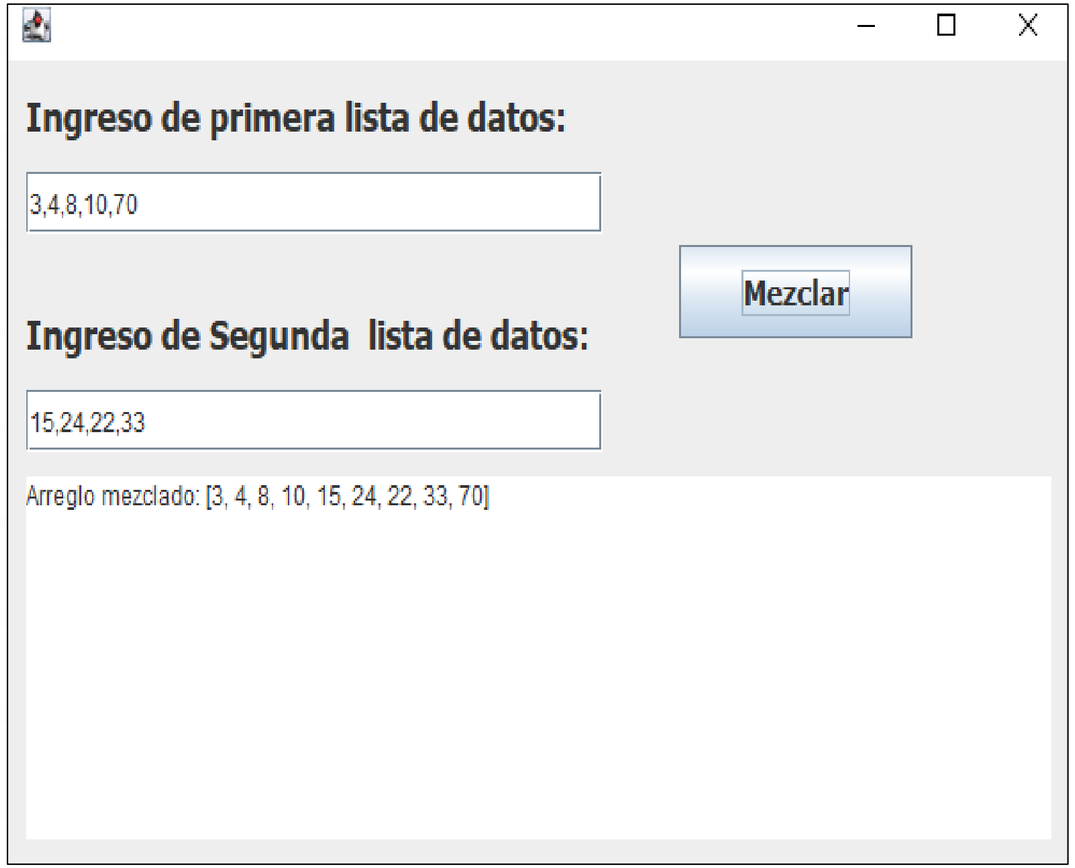


Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo 1: Mezcla Natural de dos arreglos de enteros en una interfaz gráfica





Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo 1: Mezcla Natural de dos arreglos de enteros en una interfaz gráfica

- AlgOrdExter
  - JRE System Library [JavaSE-17]
  - src
    - (default package)
    - Natural
      - AlgoOrdExNat1inter.java
      - AlgoOrdExtNat1.java
      - AlgoOrdExtNat2.java

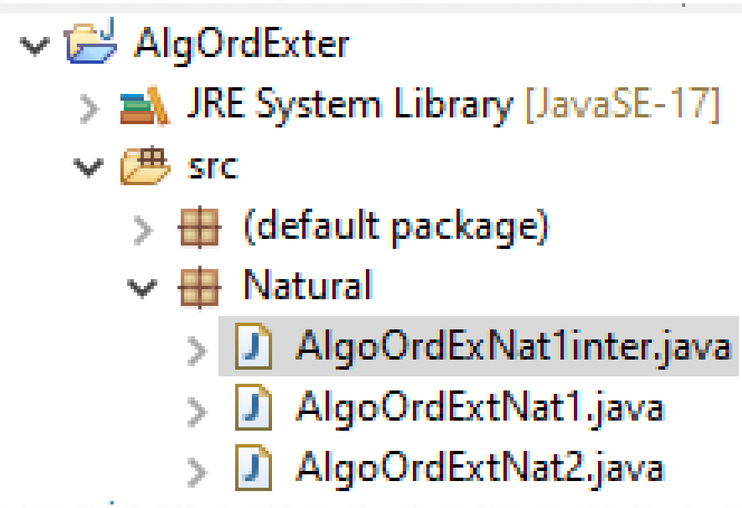
```java
public class AlgoOrdExNat1inter extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField txtDatos1;
    private JTextField txtDatos2;


    // Interfaz MezclaNatural
    interface MezclaNatural {
        int[] mezclar(int[] arreglo1, int[] arreglo2);
    }
```

```java
// Implementación de la mezcla natural
class MezclaNaturalImpl implements MezclaNatural {

    @Override
    public int[] mezclar(int[] arreglo1, int[] arreglo2) {
        int[] resultado = new int[arreglo1.length + arreglo2.length];
        int i = 0, j = 0, k = 0;

        while (i < arreglo1.length && j < arreglo2.length) {
            if (arreglo1[i] <= arreglo2[j]) {
                resultado[k++] = arreglo1[i++];
            } else {
                resultado[k++] = arreglo2[j++];
            }
        }

        while (i < arreglo1.length) {
            resultado[k++] = arreglo1[i++];
        }

        while (j < arreglo2.length) {
            resultado[k++] = arreglo2[j++];
        }

        return resultado;
    }
}
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo 1: Mezcla Natural de dos arreglos de enteros en una interfaz gráfica




```java
JButton btnMezclar = new JButton("Mezclar");
btnMezclar.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

    String[] input1 = txtDatos1.getText().split(",");
    String[] input2 = txtDatos2.getText().split(",");

    int[] arreglo1 = Arrays.stream(input1).mapToInt(Integer::parseInt).toArray();
    int[] arreglo2 = Arrays.stream(input2).mapToInt(Integer::parseInt).toArray();

    MezclaNatural mezcla = new MezclaNaturalImpl();
    int[] resultado = mezcla.mezclar(arreglo1, arreglo2);

    textArea.setText("Arreglo mezclado: " + Arrays.toString(resultado));

    }
```

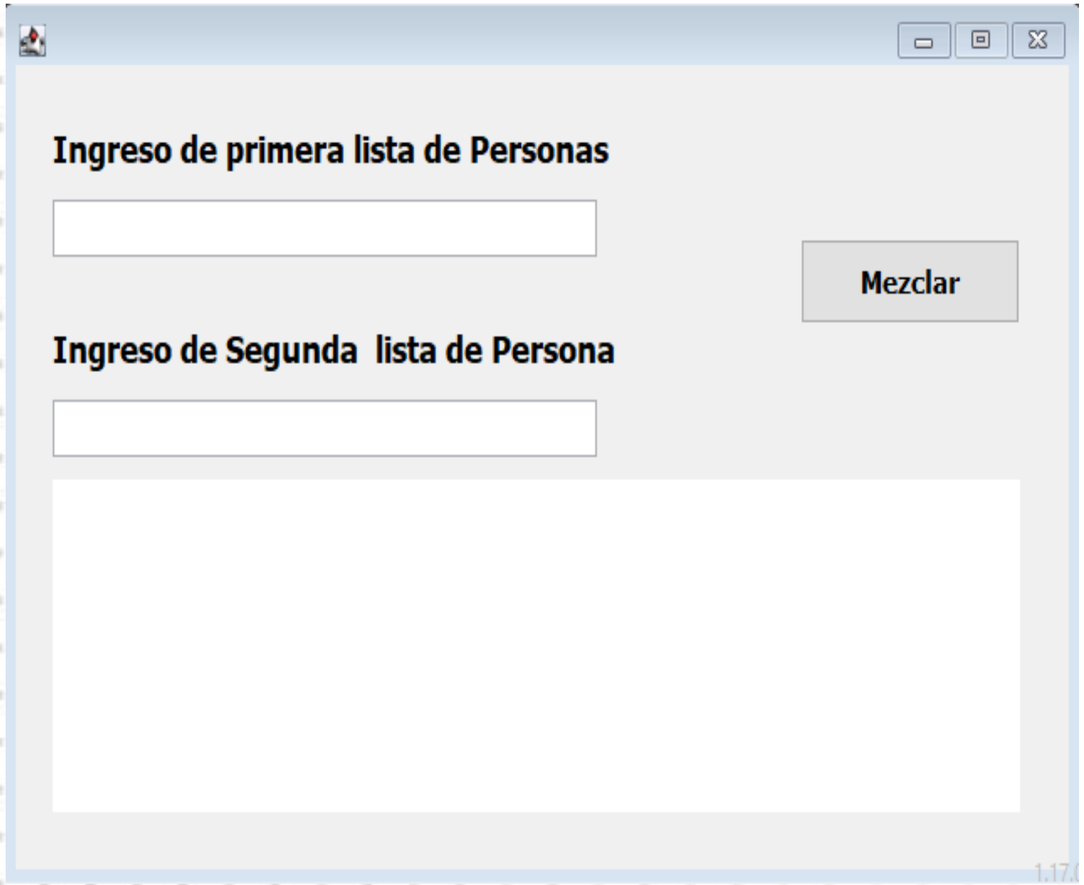
Explicación:

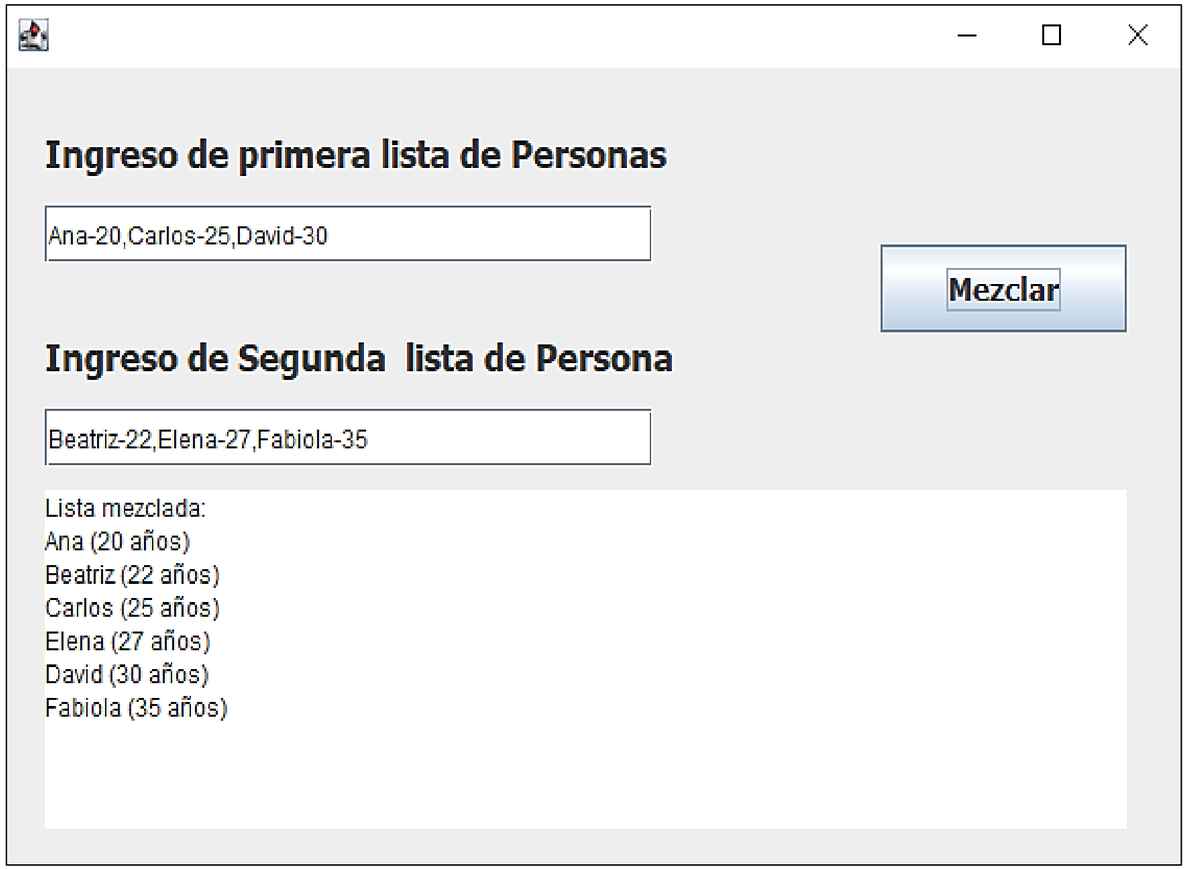
1. **Interfaz MezclaNatural**: Define el método mezclar(), que toma dos arreglos de enteros y devuelve uno mezclado de manera natural.
2. **Clase MezclaNaturalImpl**: Implementa la interfaz MezclaNatural y contiene la lógica de mezcla natural.
3. **JFrame**: El diseño usa un JFrame con dos campos de texto para ingresar los arreglos, un botón para realizar la mezcla, y un área de texto para mostrar el resultado.
4. **Acción del botón**: El botón toma las entradas de texto, las convierte en arreglos de enteros, llama al método de mezcla natural y muestra el resultado.

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo 2: Mezcla Natural de objetos (Por ejemplo, Persona) en una interfaz gráfica





Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

Ejemplo 2: Mezcla Natural de objetos (Por ejemplo, Persona) en una interfaz gráfica

- AlgOrdExter
  - JRE System Library [JavaSE-17]
  - src
    - (default package)
    - Natural
      - AlgoOrdExNat1inter.java
      - AlgoOrdExNat2inter.java
      - AlgoOrdExtNat1.java
      - AlgoOrdExtNat2.java

```java
public class AlgoOrdExNat2inter extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField txtPersonas1;
    private JTextField txtPersonas2;


    // Interfaz MezclaNatural
    interface MezclaNatural {
        List<Persona> mezclar(List<Persona> lista1, List<Persona> lista2);
    }
```

```java
// Implementación de la mezcla natural
class MezclaNaturalImpl implements MezclaNatural {

    @Override
    public List<Persona> mezclar(List<Persona> lista1, List<Persona> lista2) {
        List<Persona> resultado = new ArrayList<>();
        int i = 0, j = 0;

        while (i < lista1.size() && j < lista2.size()) {
            if (lista1.get(i).getEdad() <= lista2.get(j).getEdad()) {
                resultado.add(lista1.get(i++));
            } else {
                resultado.add(lista2.get(j++));
            }
        }

        while (i < lista1.size()) {
            resultado.add(lista1.get(i++));
        }

        while (j < lista2.size()) {
            resultado.add(lista2.get(j++));
        }

        return resultado;
    }
}
```

Universidad Tecnológica del Perú

Algoritmos de Ordenación Externa

```java
// Clase Persona
class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + " años)";
    }
}
```

```java
public void actionPerformed(ActionEvent e) {
    List<Persona> lista1 = parsePersonas(txtPersonas1.getText());
        List<Persona> lista2 = parsePersonas(txtPersonas2.getText());

        MezclaNatural mezcla = new MezclaNaturalImpl();
        List<Persona> resultado = mezcla.mezclar(lista1, lista2);

        textArea.setText("Lista mezclada:\n");
        for (Persona persona : resultado) {
            textArea.append(persona.toString() + "\n");
        }
}

private List<Persona> parsePersonas(String input) {
    return Arrays.stream(input.split(","))
            .map(persona -> persona.split("-"))
            .map(parts -> new Persona(parts[0], Integer.parseInt(parts[1])))
            .collect(Collectors.toList());
}
```

Explicación:

1. **Interfaz MezclaNatural**: Define el método mezclar(), que toma dos listas de Persona y devuelve una lista combinada.
2. **Clase Persona**: Representa a una persona con un nombre y edad.
3. **JFrame**: La interfaz gráfica contiene campos de texto para que el usuario ingrese dos listas de personas (en formato nombre-edad), un botón para realizar la mezcla y un área de texto para mostrar los resultados.
4. **Parseo de entrada**: El texto ingresado se convierte en objetos Persona a partir de un formato nombre-edad mediante el método parsePersonas().
5. **Acción del botón**: Al presionar el botón, se mezcla la lista de personas de acuerdo con la edad y se muestra el resultado en un área de texto.

Universidad Tecnológica del Perú

Universidad
Tecnológica
del Perú